

Cockpit Knowage TransportDakar

Guide technique — Navigation, Mode clair/sombre, Graphiques à axes réels

Ibtihel Abdellaoui — PFE ESPRIT / CETUD

Juin 2026

Contents

1	Contexte	2
2	1. Navigation entre les sheets	3
2.1	1.1 Deux niveaux de navigation, complémentaires	3
2.2	1.2 Recette pratique pour ajouter/relier un sheet	3
3	2. Mode clair / sombre (100 % CSS, sans JavaScript)	5
3.1	2.1 Le principe : case à cocher invisible + sélecteur CSS :has()	5
3.2	2.2 Méthode pour répliquer ce mode sur un nouveau sheet	6
4	3. Graphiques à barres avec axe Y réel et étiquettes de valeur	7
4.1	3.1 Le problème initial	7
4.2	3.2 La solution : axe Y gradué + étiquette au-dessus de chaque barre . . .	7
4.3	3.3 Méthode pour appliquer ce patron à un nouveau graphique	8
5	4. Déploiement et vérification dans Knowage (rappel pratique)	10
5.1	4.1 Les deux niveaux de sauvegarde	10
5.2	4.2 Procédure recommandée à chaque modification	10

1 Contexte

Le cockpit Knowage TransportDakar est un document composite accessible à l'URL `workspace/document-composite/Accueil`, composé de six feuilles (sheets) : **Accueil, Trafic, Déplacements, Démographie, Accessibilité, IA & Prévisions**.

Chaque sheet est un widget HTML/CSS unique, géré dans l'éditeur de widget de Knowage. Une contrainte technique gouverne tout ce document : **Knowage applique un filtre de sécurité (sanitizer) côté serveur** au moment de la sauvegarde au niveau du document. Ce filtre rejette systématiquement :

- toute balise `<script>` ;
- tout attribut `onclick`, `onchange`, etc. (tout gestionnaire d'événement inline).

Si l'une de ces deux choses est présente, la sauvegarde échoue avec le message « **Invalid HTML payload** », même si la sauvegarde « locale » du widget (bouton SAVE dans l'éditeur) avait semblé réussir.

Conséquence directe : la navigation, le mode clair/sombre et les graphiques ont tous été conçus pour fonctionner **uniquement avec du HTML et du CSS**, sans une seule ligne de JavaScript. C'est la clé de voûte des trois solutions décrites dans ce document.

2 1. Navigation entre les sheets

2.1 1.1 Deux niveaux de navigation, complémentaires

2.1.1 a) La navigation native de Knowage (barre d'onglets)

Knowage affiche nativement, en haut du document composite, une barre d'onglets correspondant à chaque sheet créé. C'est le mécanisme de navigation **principal et fonctionnel** : cliquer sur un onglet change immédiatement de sheet. Il ne nécessite aucun code — il suffit que les six sheets existent dans le document composite.

C'est ce mécanisme qui a été testé et validé pour circuler entre Accueil, Trafic, Déplacements, Démographie, Accessibilité et IA, dans les deux sens.

2.1.2 b) Les « cartes de navigation » visuelles à l'intérieur de chaque sheet

En complément, chaque sheet contient en bas de page un bloc de cartes stylisées reproduisant visuellement les six destinations, avec icône et libellé, et une mise en évidence (couleur orange + fond clair) sur la carte correspondant au sheet actuellement affiché :

```
<div class="td-nav">
  <div class="td-navcard"><div class="ic">□</div><div class="lb">Trafic</div></div>
  <div class="td-navcard"><div class="ic">□</div><div class="lb">Déplacements</div></div>
  <div class="td-navcard"><div class="ic">□</div><div class="lb">Démographie</div></div>
  <div class="td-navcard"><div class="ic">□</div><div class="lb">Accessibilité</div></div>
  <div class="td-navcard active"><div class="ic">□</div><div class="lb">IA</div></div>
</div>
```

```
.td-nav { display:grid; grid-template-columns:repeat(5,1fr); gap:12px; }
.td-navcard {
  background:#fff; border:1px solid #E2E8F0; border-radius:12px;
  padding:16px 10px; text-align:center; box-shadow:0 2px 8px rgba(15,23,42,.05);
  cursor:pointer; transition:.2s;
}
.td-navcard:hover {
  border-color:#F97316; box-shadow:0 6px 16px rgba(249,115,22,.18);
  transform:translateY(-3px);
}
.td-navcard.active { border-color:#F97316; background:#FFF7ED; }
```

Point important : ces cartes sont **purement visuelles**. Elles n'ont pas d'attribut onclick (interdit par le sanitizer) et ne déclenchent donc pas elles-mêmes le changement de sheet — elles servent de repère visuel cohérent sur toutes les pages (« vous êtes ici »), tandis que le changement de sheet réel se fait via la barre d'onglets native de Knowage. C'est la seule classe `.active` qui change d'un sheet à l'autre dans le code HTML déployé sur chaque sheet.

2.2 1.2 Recette pratique pour ajouter/relier un sheet

1. Construire le widget HTML/CSS du nouveau sheet en suivant le même gabarit (entête, KPI, graphiques, bloc `.td-nav`).

2. Dans le bloc `.td-nav`, ajouter la classe active uniquement sur la carte correspondant à ce sheet.
3. Déployer le widget (voir section 4).
4. Vérifier dans la barre d'onglets Knowage que le nouveau sheet apparaît et qu'il est atteignable depuis les autres sheets.

3 2. Mode clair / sombre (100 % CSS, sans JavaScript)

3.1 2.1 Le principe : case à cocher invisible + sélecteur CSS :has()

Puisque le JavaScript est interdit par le sanitizer, le bouton de bascule clair/sombre repose entièrement sur une astuce CSS native : un `<input type="checkbox">` caché, couplé à un `<label>` cliquable, et le sélecteur CSS `:has()` qui permet de styliser un parent en fonction de l'état d'un enfant.

3.1.1 a) Le HTML (un seul exemplaire, placé dans l'en-tête de chaque sheet)

```
<input type="checkbox" id="td-dark-toggle" style="display:none">
<label for="td-dark-toggle" class="td-theme-btn" title="Basculer mode clair / sombre"></label>
```

- La case à cocher est masquée (`display:none`) mais reste interactive : cliquer sur le `<label>` (grâce à l'attribut `for`) coche/décoche la case, sans aucun script.
- Le bouton visible à l'écran est en réalité le `<label>`, stylé pour ressembler à un bouton rond.

3.1.2 b) Le style du bouton

```
.td-theme-btn {
  margin-left:auto; background:rgba(255,255,255,.1);
  border:1px solid rgba(255,255,255,.25); color:#fff;
  border-radius:10px; width:38px; height:38px; font-size:17px;
  cursor:pointer; display:flex; align-items:center; justify-content:center;
  transition:.2s; flex-shrink:0;
}
.td-theme-btn:hover { background:rgba(249,115,22,.35); border-color:#F97316; }
```

3.1.3 c) Le cœur du mécanisme : :has()

/ Tant que la case n'est pas cochée -> thème clair par défaut, aucune règle supplémentaire */*

/ Dès que la case EST cochée -> on cible le conteneur racine du sheet */*

```
#td-home:has(#td-dark-toggle:checked) {
  background:#0B1220;
  color:#E2E8F0;
}
#td-home:has(#td-dark-toggle:checked) .td-kpi {
  background:#16213A; border-color:#28344F;
  box-shadow:0 2px 8px rgba(0,0,0,.35);
}
#td-home:has(#td-dark-toggle:checked) .td-kpi .val { color:#F1F5F9; }
#td-home:has(#td-dark-toggle:checked) .td-kpi .lbl { color:#94A3B8; }
#td-home:has(#td-dark-toggle:checked) .td-card {
  background:#16213A; border-color:#28344F;
  box-shadow:0 2px 8px rgba(0,0,0,.35);
}
#td-home:has(#td-dark-toggle:checked) .td-card h3 { color:#F1F5F9; }
```

```

#td-home:has(#td-dark-toggle:checked) .td-resume li { border-bottom-color:#28344F; }
#td-home:has(#td-dark-toggle:checked) .td-resume li b { color:#F1F5F9; }
#td-home:has(#td-dark-toggle:checked) .td-barlabels span { color:#94A3B8; }
#td-home:has(#td-dark-toggle:checked) .td-yaxis { color:#64748B; border-right-color:#28344F; }
#td-home:has(#td-dark-toggle:checked) .td-bars {
  background-image:repeating-linear-gradient(180deg,#1E293B 0,#1E293B 1px,transparent 1px);
}
#td-home:has(#td-dark-toggle:checked) .bar .bv { color:#F1F5F9; }
#td-home:has(#td-dark-toggle:checked) .td-navcard { background:#16213A; border-color:#28344F; }
#td-home:has(#td-dark-toggle:checked) .td-navcard .lb { color:#F1F5F9; }
#td-home:has(#td-dark-toggle:checked) .td-navcard.active { background:#1E2A4A; border-color:#28344F; }

```

Comment le lire : `#td-home:has(#td-dark-toggle:checked)` signifie « le conteneur `#td-home`, à condition qu’il contienne quelque part un élément `#td-dark-toggle` qui est `:checked` ». Tant que l’utilisateur n’a pas cliqué, la case n’est pas cochée, cette condition est fausse, et aucune des règles sombres ne s’applique — le sheet reste dans son thème clair par défaut. Dès le clic, la case passe à `checked`, la condition devient vraie, et **toutes** les règles préfixées par ce sélecteur s’appliquent instantanément, sans rechargement de page ni script.

3.2 2.2 Méthode pour répliquer ce mode sur un nouveau sheet

1. Donner à la `<div>` racine du sheet un identifiant unique, ex. `#td-home` (le même identifiant doit être réutilisé dans toutes les règles `:has()`).
2. Ajouter une fois la case à cocher + le label dans l’en-tête.
3. Pour chaque composant visuel du sheet (cartes KPI, graphiques, listes, cartes de navigation...), écrire la règle sombre correspondante en préfixant son sélecteur normal par `#td-home:has(#td-dark-toggle:checked)`.
4. Ne jamais utiliser `onclick`, `onchange` ou une balise `<script>` — uniquement `<input>` + `<label for="...">` + CSS.

4 3. Graphiques à barres avec axe Y réel et étiquettes de valeur

4.1 3.1 Le problème initial

Dans la première version des sheets, chaque barre était positionnée uniquement avec une hauteur en pourcentage (`style="height:62%"`), sans aucune indication numérique ni repère sur l'axe vertical. Impossible de savoir si 62 % correspondait à 62 trajets, 62 % d'un score, ou autre chose, et impossible de lire précisément la valeur d'une barre sans deviner.

4.2 3.2 La solution : axe Y gradué + étiquette au-dessus de chaque barre

4.2.1 a) Structure HTML d'un graphique

```
<div class="td-chart">
  <div class="td-yaxis">
    <span>100</span><span>75</span><span>50</span><span>25</span><span>0</span>
  </div>
  <div class="td-barswrap">
    <div class="td-bars">
      <div class="bar" style="height:55%"><span class="bv">55%</span></div>
      <div class="bar" style="height:62%"><span class="bv">62%</span></div>
      <div class="bar" style="height:58%"><span class="bv">58%</span></div>
      <div class="bar" style="height:70%"><span class="bv">70%</span></div>
      <div class="bar" style="height:85%"><span class="bv">85%</span></div>
      <div class="bar" style="height:95%"><span class="bv">95%</span></div>
      <div class="bar" style="height:48%"><span class="bv">48%</span></div>
    </div>
    <div class="td-barlabels">
      <span>Lun</span><span>Mar</span><span>Mer</span><span>Jeu</span><span>Ven</span><span>Sam</span><span>Dim</span>
    </div>
  </div>
</div>
```

Trois blocs distincts, côte à côte :

1. **.td-yaxis** — la colonne d'échelle verticale, avec les graduations (ici 100/75/50/25/0, à adapter à l'échelle réelle des données).
2. **.td-barswrap > .td-bars** — les barres elles-mêmes, chacune contenant un `<span class="bv">` (« bar value ») affichant la vraie valeur numérique, positionné juste au-dessus de la barre.
3. **.td-barlabels** — les libellés de l'axe horizontal (jours, catégories, etc.).

4.2.2 b) CSS correspondant

```
.td-chart { display:flex; gap:10px; }

.td-yaxis {
  width:40px; flex-shrink:0; display:flex; flex-direction:column;
```

```

    justify-content:space-between; align-items:flex-end; height:150px;
    padding-right:6px; border-right:1px dashed #CBD5E1;
    font-size:9px; color:#94A3B8; font-weight:700;
}

.td-barswrap { flex:1; min-width:0; }

.td-bars {
    display:flex; align-items:flex-end; gap:6px; height:150px; margin-top:6px;
    position:relative;
    background-image:repeating-linear-gradient(180deg,#EEF2F7 0,#EEF2F7 1px,transparent 1px);
}

.td-bars .bar {
    flex:1; position:relative;
    background:linear-gradient(180deg,#F97316,#0F172A);
    border-radius:4px 4px 0 0;
}

.td-bars .bar .bv {
    position:absolute; left:0; right:0; bottom:100%; margin-bottom:4px;
    text-align:center; font-size:9px; font-weight:700; color:#0F172A;
    white-space:nowrap;
}

.td-barlabels { display:flex; gap:6px; margin-top:6px; }
.td-barlabels span { flex:1; text-align:center; font-size:9.5px; color:#64748B; }

```

4.2.3 c) Pourquoi ça fonctionne sans JavaScript

- height: NN% reste une simple valeur CSS inline calculée à l'avance (côté contenu, pas côté script) — chaque barre a sa propre hauteur, propre à la donnée qu'elle représente.
- L'axe Y est un texte statique : il suffit de calculer manuellement les graduations cohérentes avec la donnée maximale du graphique (ex. si la donnée max est 95 %, une échelle 100/75/50/25/0 convient ; si la donnée max est 14, une échelle 15/10/5/0 conviendrait mieux).
- L'étiquette de valeur .bv est positionnée en CSS pur via position:absolute; bottom:100%, ce qui la place toujours juste au-dessus du sommet de sa barre, quelle que soit la hauteur de celle-ci.
- Le fond hachuré (repeating-linear-gradient) en arrière-plan des barres simule un quadrillage de lecture à 25/50/75/100 %, sans avoir besoin de dessiner des lignes en JavaScript ou en SVG.

4.3 3.3 Méthode pour appliquer ce patron à un nouveau graphique

1. Identifier la donnée maximale de la série à représenter.
2. Choisir une échelle ronde et lisible (0/25/50/75/100, ou 0/5/10/15/20, etc. selon l'ordre de grandeur).

3. Remplir `.td-yaxis` avec les graduations dans l'ordre décroissant (la plus grande valeur en haut).
4. Pour chaque catégorie, créer une `.bar` avec `style="height:X%"` (X = valeur réelle exprimée en pourcentage de l'échelle choisie) et un `<span class="bv">` contenant la vraie valeur à afficher (ex. « 62 % », « 316 », « 28 min »...).
5. Reproduire le bloc `.td-barlabels` avec les libellés de chaque catégorie, dans le même ordre que les barres.

5 4. Déploiement et vérification dans Knowage (rappel pratique)

Ce point conditionne la réussite des trois sections précédentes : un code parfaitement correct peut être rejeté ou perdu si la procédure de sauvegarde n'est pas respectée.

5.1 4.1 Les deux niveaux de sauvegarde

Niveau	Action	Fiabilité
Widget	Bouton SAVE dans l'éditeur HTML du widget	Accepté visuellement à chaque fois, sans aucune validation de fond — ne garantit rien
Document	Icône disque dans le menu cockpit (hamburger)	Seule sauvegarde qui déclenche le filtre de sécurité serveur — confirmation par un toast « Saved », ou rejet avec « Invalid HTML payload »

5.2 4.2 Procédure recommandée à chaque modification

1. Ouvrir l'éditeur HTML du widget du sheet concerné.
2. Remplacer le contenu par le nouveau code (HTML + CSS), en s'assurant qu'il ne contient **aucune** balise `<script>` ni attribut `onclick/onchange`.
3. Cliquer sur **SAVE** dans l'éditeur de widget.
4. Ouvrir le menu cockpit (icône hamburger) puis cliquer sur l'icône **disque** pour sauvegarder au niveau du document.
5. Vérifier l'apparition du toast « **Saved** » (succès) plutôt que « **Invalid HTML payload** » (échec — revenir au code et retirer tout JavaScript/attribut interdit).
6. **Recharger complètement la page** (pas un simple changement d'onglet) et vérifier visuellement que la modification est bien présente. C'est la seule façon de confirmer que la sauvegarde a réellement persisté côté serveur, et non simplement dans l'état temporaire de l'éditeur.

